

Autonomous Agents and Intelligent Robotics

Giorgio Battistelli

Department of Information Engineering, University of Florence, Italy



UNIVERSITÀ
DEGLI STUDI
FIRENZE

DINFO
DIPARTIMENTO DI
INGEGNERIA
DELL'INFORMAZIONE

Autonomous Agents and Intelligent Robotics

- Class schedule:
 - Thursday 10:15-13:15 (Morgagni 008)
 - Friday 8:15-10:15 (S. Marta 033)
- In-person or Meet consultations
 - Appointment via email
- Oral exam

Learning Objectives

Learning Objectives: You will learn how to model, analyze, and design intelligent agents able to carry out complex tasks, taking into account:

- both physical and virtual agents (autonomous vehicles, robots, sensors, processing units, pieces of software)
- decentralized architectures
- interacting, possibly heterogeneous, agents

- 1 **Intelligent agents**
- 2 **Robot motion planning**
- 3 **Elements of graph theory**
- 4 **Synchronization and coordination in Multi-agent systems**
- 5 **Multi-robot systems**
- 6 **Multi-agent optimization and learning**
- 7 **Reinforcement Learning**

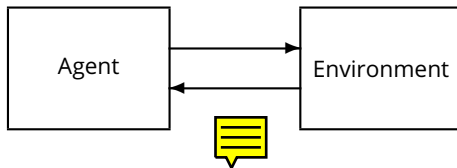
Suggested readings

- S. Russell, P. Norvig, *Artificial intelligence: a modern approach*, 2016 (topics 1, 2, 7)
- M. Wooldridge, *An Introduction to Multiagent Systems*, 2009 (topic 1)
- S. La Valle, *Planning algorithms*, 2006 (topics 2, 7)
- M. Newman, *Networks: an Introduction*, (topic 3)
- F. Bullo, *Lectures on Network Systems*, 2018 (topics 3, 4)
- M. Mesbahi, M. Egerstedt, *Graph Theoretic Methods in Multiagent Networks*, 2010 (topics 3,4,5)
- R. Sutton and A. Barto, *Reinforcement Learning: an Introduction*, 2018 (topic 7)
- D. Bertsekas, J. Tsitsiklis, *Neuro-dynamic programming*, 1996 (topic 7)
- Selected articles

1 Intelligent agents

1.1 **Intelligent agents:** Agent model

What is an agent?



Agent: anything that can interact with the environment

- receive *data/information* from the environment, for example through *sensors*
 - *act* on the environment, for example through *effectors*
-
- An agent can be classified as
 - *physical*: a human, an animal, a robot, an autonomous vehicle, etc.
 - *virtual*: a piece of software

Agent-environment interaction

- t is a time index
- $x_a(t)$ state/configuration of the agent at time t
- $x_e(t)$ state/configuration of the environment at time t
- $x(t) = (x_a(t), x_e(t))$ state at time t
- $u(t)$ action of the agent at time t

To model the interaction between agent and environment

$$x(t+1) = f(x(t), u(t))$$

the action of the agent can modify the states of both the environment and the agent itself

State and action spaces

- $x(t)$ belongs to the state space $\mathbb{X}$
- $u(t)$ belongs to the action space $\mathbb{U}$
- the sets $\mathbb{X}$ and $\mathbb{U}$ can be
 - continuous – typical in control theory
 - discrete – typical in artificial intelligence
 - hybrid – for example in autonomous driving the action is both continuous (steering, acceleration) and discrete (gear)

Remark: the same agent can be modelled as continuous or discrete depending on the situation (for example: robot navigation)

Example: mobile robot – continuous model

- **Basic motion model for mobile robots**

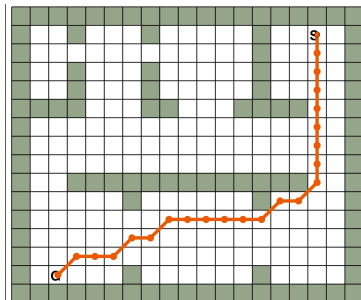
$$x(t + 1) = x(t) + T_s u(t)$$



- T_s is the sampling time
- $x(t)$ is the robot position at time t :
 - $x \in \mathbb{R}^2$ for planar (2D) motion (for example, autonomous *ground* vehicles)
 - $x \in \mathbb{R}^3$ for 3D motion (autonomous *air* or *underwater* vehicles)
- $u(t)$ is the velocity vector (it can be designed so as to assign to the robot a desired behaviour)

Example: mobile robot – discrete model

- For navigation purposes: sometimes useful to discretize the map by gridding
- $x(t)$ is the cell occupied by the robot at time t
- $u(t)$ is the direction of motion



- Action space

$$\mathbb{U} = \{(0, 0), (1, 0), (0, 1), (-1, 0), (0, -1), (1, 1), (1, -1), (-1, 1), (-1, -1)\}$$

Remark: depending on the map and the robot location, some actions are not allowed

Intelligent agents

Question: what makes an agent intelligent?



Intelligent agents

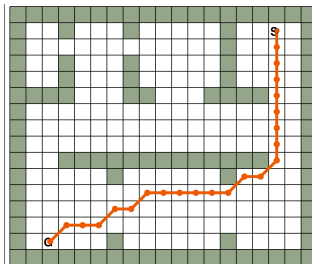
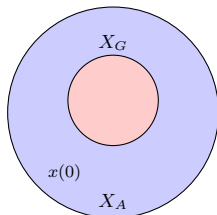
Question: what makes an agent intelligent?

- Intelligent agents typically exhibit some of the following characteristics:
 - **autonomous:** able to operate without direct intervention and control their own actions/decisions
 - **goal-oriented:** acting in order to achieve specific objectives
 - **adaptive:** capable of adjusting their behaviour to changes in the environment
 - **robust:** able to maintain functionality in the presence of disturbances or uncertainty
 - capable of **learning** from past experience (i.e., collected data)
- An intelligent agent decides its actions based on
 - the defined goal/objective
 - the available information (possibly incomplete or uncertain)

Objective specification

Given a time horizon T (either finite or infinite), the agent's objectives can be defined in terms of

- A set $X_G \subset \mathbb{X}$ of **desired configurations**
- A set $X_A \subset \mathbb{X}$ of **admissible configurations**
- A **reward function** R to maximize
(or equivalently a **cost function** J to minimize)



Objective specification

In practice, objectives are often **multi-objective** (e.g., performance, energy efficiency, safety) and may require balancing trade-offs.

- **Objective in terms of desired configurations:**

the state $x(t)$ is required to reach the set X_G

- Finite horizon T :

$$x(T) \in X_G$$

- Infinite horizon:

$$\text{dist}(x(t), X_G) \rightarrow 0 \quad \text{as } t \rightarrow \infty$$



- **Objective in terms of admissible configurations:**

assuming that the initial state is feasible, $x(0) \in X_A$, the state must remain feasible for all times:

$$x(t) \in X_A \quad \text{for } 0 \leq t \leq T$$

(These correspond to reachability and safety specifications, respectively)

Reward function

- **Instantaneous reward** at time t : the received reward depends on the state in which we are and on the action we take

$$r(t, x(t), u(t))$$

- **Total reward** also called **return** or **utility**:



$$R = \sum_{t=0}^{T-1} r(t, x(t), u(t))$$

- The total reward depends on
 - the initial state $x(0)$
 - the sequence of decisions $u(0), u(1), \dots, u(T-1)$

Decision policy

The agent selects its actions based on the available information

$$u(t) = \pi(x(t)) \quad \text{or more generally} \quad u(t) = \pi(t, x(t))$$

- In the artificial intelligence literature, π is called a **policy** (or **decision policy**)
- In the control literature, π corresponds to the **control law**

Problem: devise a policy π that enables the agent to achieve its objectives

Markov transition density

- In many situations, state transitions from time t to time $t + 1$ are not deterministic.

A probabilistic transition can be modelled through the **Markov transition density**

$$\varphi(x(t+1) | x(t), u(t))$$



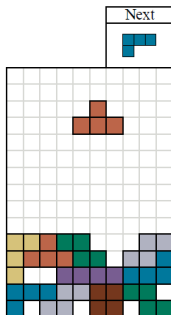
which defines the probability of transitioning from state $x(t)$ to state $x(t+1)$ when action $u(t)$ is applied.

- In this case, the **expected return** with respect to uncertain transitions is

$$R = E \left\{ \sum_{t=0}^{T-1} r(t, x(t), u(t)) \right\}$$

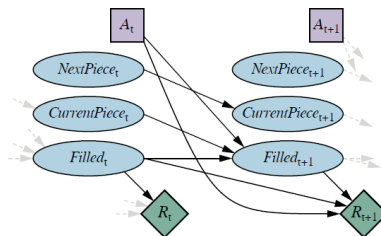
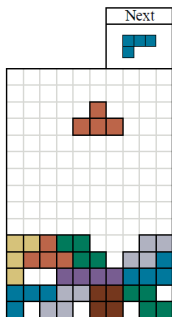
- The resulting framework is known as a **Markov decision process (MDP)**.
- MDPs are the standard mathematical framework for **reinforcement learning**.

Example: Tetris game



- State?
- Action?
- State transition?
- Reward?

Example: Tetris game



- State = (board configuration, current piece, next piece)
- Action = where to place the current piece
- Markov transition because next piece chosen randomly from the 7 piece types
- Terminal state = when the board fills up to the top (game over)
- Reward = game score (points earned for cleared lines)

Observation model

Observation model: describes the information gathered by the agent at time t

$$y(t) = h(x(t))$$

- The observation $y(t) \in \mathbb{Y}$ may also depend on the previous action $u(t-1)$
- The observation space $\mathbb{Y}$ can be continuous, discrete, or hybrid
- **Complete information:** the entire state is observed, $y(t) = x(t)$
- **Partial information:** only partial information about the state is available (e.g., a robot perceives only noisy sensor data)

Information vector

- **Information vector:** summarizes all the information collected up to time t

$$I(t) = \begin{bmatrix} y(0) \\ u(0) \\ y(1) \\ \vdots \\ u(t-1) \\ y(t) \end{bmatrix}$$

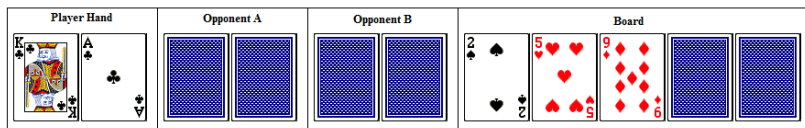
- The agent selects its actions based on the available information

$$u(t) = \pi(t, I(t))$$

- Under full observability and Markov dynamics, the current state $x(t)$ is a **sufficient statistic** for the information history $I(t)$. Then we can write

$$u(t) = \pi(t, x(t))$$

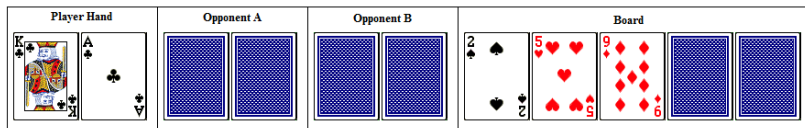
Example: poker



- **Hidden / private information:** each player holds a set of cards (their *hand*) that is hidden from opponents.
- This private information makes the game **partially observable**, since a player can only see their own hand, the community cards (in variants like Texas Hold'em), and the betting history.

Question: which observations does each player use to decide its actions?

Poker: observations available to each player



Question: which observations does each player use to decide its actions?

- **Observation $y(t)$:** what the player perceives at time t
 - own (private) cards
 - visible (community) cards
 - current bets on the table
- **Information vector $I(t)$:** collects all past observations and actions up to time t
 - own (private) cards
 - sequence of revealed community cards
 - complete betting history

Likelihood function

- In many situations, the observation model is **not deterministic!**
- **Example:** a range sensor such as LiDAR or sonar provides noisy distance measurements!

A probabilistic observation can be modelled through the **likelihood function**

$$\ell(y(t)|x(t))$$

probability of observing $y(t)$ given that the state is $x(t)$

- In this case the considered framework is known as **Partially observable MDP (POMDP)**

1.2 Intelligent agents: Multi-agent systems

What is a multi-agent system?

Multi-agent system (MAS): a system of multiple intelligent agents that interact with one another and with the environment.

- In this course, we focus on **decentralized** multi-agent systems:
 - each agent makes its own decisions **independently** based on locally available information;
 - this information may be either **complete or incomplete information**;
 - agents affect one another through the shared environment, coupled dynamics, **sensing**, or communication.



Individualistic, competitive, and cooperative MAS

- Multi-agent systems can be classified as:
 - **Individualistic:** agents pursue separate objectives that are neither shared nor directly conflicting;
 - **Competitive:** agents pursue partially or fully conflicting objectives;
 - **Cooperative:** agents share a common goal or objective.



Competition



Cooperation

Agents-environment interaction

- N number of agents
- $x_i(t)$ state/configuration of agent i at time t
- $x_e(t)$ state/configuration of the environment at time t
- $x(t) = (x_1(t), \dots, x_N(t), x_e(t))$ state at time t
- $u_i(t)$ action of agent i at time t

To model how the agents interact with each other and with the environment

$$x(t+1) = f(x(t), u_1(t), \dots, u_N(t))$$

Observation model

Observation model: describes the information available to agent i at time t

$$y_i(t) = h_i(x(t))$$

- **Information vector:** collects all the information available to agent i up to time t

$$I_i(t) = (y_i(0), u_i(0), y_i(1), \dots, u_i(t-1), y_i(t))$$

Remark: in general, different agents have different information vectors!

Example of observation model

- $\mathcal{N} = \{1, 2, \dots, N\}$ set of agents
- Each agent i can observe
 - the local state $x_i(t)$
 - the states $x_j(t)$ of a subset of agents $\mathcal{N}_i \subseteq \mathcal{N} \setminus \{i\}$
- $\mathcal{N}_i$ is the set of **neighbors** of agent i
- **Example:** in a vehicle platoon, each car observes its own velocity and position, and those of the vehicles immediately ahead and behind

Observation model: to model the information gathered by agent i at time t

$$y_i(t) = \begin{bmatrix} x_i(t) \\ x_j(t), \quad j \in \mathcal{N}_i \end{bmatrix}$$



Remark: the information exchange between agents defines a **network** or **graph**

Decentralized decisions

Each agent i selects its actions based on the locally available information

$$u_i(t) = \pi_i(t, I_i(t)) \quad i = 1, \dots, N$$

- π_i is the decision policy of agent i ;
- decisions are decentralized because each agent autonomously selects its own action;
- although decisions are decentralized, the agents remain coupled through their dynamics, objectives, and information exchange.

Problem: determine the local policies $\pi_1, \dots, \pi_N$ that enable the agents to achieve their individual or common objectives, subject to their information constraints.

Objective specification

Given a time horizon T (either finite or infinite), the objectives of agent i can be defined in terms of

- A set $X_{G,i} \subset \mathbb{X}$ of **desired configurations**
- A set $X_{A,i} \subset \mathbb{X}$ of **admissible configurations**
- A **reward function**

$$R_i = \sum_{t=0}^{T-1} r_i(t, x(t), u_1(t), \dots, u_N(t))$$



to be maximized

- $r_i(t, x(t), u_1(t), \dots, u_N(t))$ reward received by agent i at time t can also depend on the (unknown) actions of the other agents

Cooperative vs competitive MAS

- **Cooperative MAS:** all agents have common objectives

$$X_{G,i} = X_G \quad i = 1, \dots, N$$

$$X_{A,i} = X_A \quad i = 1, \dots, N$$

$$r_i = r \quad i = 1, \dots, N$$

- **Competitive MAS:** agents have conflicting objectives

A fully competitive two agent zero-sum example is:

$$X_{G,1} \cap X_{G,2} = \emptyset$$

$$r_2 = -r_1$$

1.3 **Intelligent agents:** Paradigms in intelligent agents

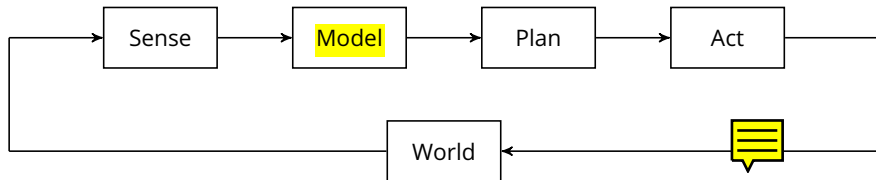
Deliberative vs reactive agents

There are two main paradigms for creating intelligent agents:

- **Deliberative agents:** based on **world models** and **planning**
 - **Reactive agents:** based on **behaviors**
-
- The first paradigm was historically popular in the AI community
 - The second paradigm was historically popular in the robotics community
 - The two paradigms can be combined to obtain **hybrid agents** (common nowadays in robotics)

Deliberative agents

- The deliberative agent paradigm is inspired by advanced cognitive procedures displayed in humans
- Deliberative agents follow the **sense-plan-act** approach
 - **Sense:** perceive the world (environment + agents)
 - **Model:** update the internal model of the world
 - **Plan:** select an action by **reasoning** about the effects of various actions on the world model
 - **Act:** apply the selected action and change the world



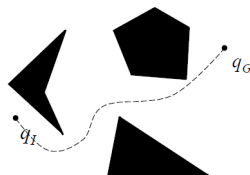
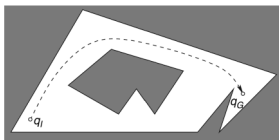
Example: robot motion planning

Robot motion planning: given

- a *start* pose of the robot
- a desired *goal* pose
- a geometric description of the robot and the environment

Find a path that

- moves the robot from start to goal as fast as possible
- *avoids collisions* with the obstacles



Limitations of purely deliberative architectures

Deliberative agents can achieve high performance when the environment is sufficiently well modeled and the planning problem is tractable.

- **Model dependence:** an accurate world model may be difficult to obtain and can rapidly become outdated in uncertain or dynamic environments
- **Computational burden:** planning over long horizons or large state and action spaces can be computationally demanding (*curse of dimensionality*)
- **Limited responsiveness:** unexpected events occurring during plan execution may require frequent state updates and replanning

These limitations motivate the integration of deliberative planning with reactive components in hybrid architectures.

Reactive agents

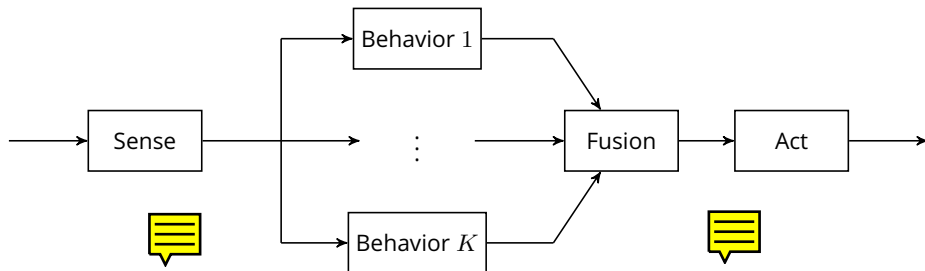
Reactive agents build complex behavior from simple components

- The behavioral paradigm was introduced by Rodney Brooks as an alternative to classical AI (Elephants don't play chess, 1990)
- According to this paradigm, **real intelligence is situated in the world, not in disembodied systems such as theorem provers or expert systems**
⇒ **behavior-based robotics**
- Main ideas:
 - Intelligent behavior can be generated **without explicit representation of the world**
 - Intelligent behavior can be generated **without explicit abstract reasoning** (planning)
 - Intelligence is an **emergent** property of certain complex systems

Behaviors

Behavior: a rather simple policy (control law) to accomplish a single task/ achieve a single goal (usually simple)

- The action is obtained by combining/fusing different behaviors
- For example in robot navigation possible behaviors are: go towards the goal, avoid collisions, explore the environment, etc.



Behavior coordination

- Behaviors can be coordinated in different ways:
 - **Fusion:** combine the outputs of multiple behaviors, for example through a weighted sum
 - **Arbitration:** select one behavior among the currently active ones
 - **Layered coordination:** organize behaviors according to different levels of competence
- **Subsumption architecture:** a *hierarchy* of task-accomplishing behaviors:
 - each layer implements a relatively simple behavior
 - behaviors are organized according to increasing levels of competence
 - lower layers typically implement more primitive behaviors, such as obstacle avoidance

Emergence

- Putting simple behaviors together leads to *synergies*
- When observed behavior exceeds programmed behavior, then we have **emergence** (the whole is greater than the sum of its parts)
- Emergence is typical in biological multi-agent systems (possibly also in the human brain)
- For example in nature, social animals and birds align themselves in swarms, schools, flocks, and herds



Limitations of reactive agents

Remark: Reactive agents have advantages but also limitations!

- reactive agents need a lot of information from the *sensors*, since they do not have a model of the world
- since actions are based on local information, such agents inherently take a *short-term* view
- emergent behavior is very *hard to engineer* or validate; typically a *trial and error* approach is adopted
- models using many layers are inherently complex and difficult to understand

Hybrid agents

Hybrid agents try to combine the speed of reactive agents with the power of deliberative agents

- In many cases, neither a completely deliberative nor a completely reactive approach is suitable for building agents
- Hybrid agents are built out of two (or more) subsystems:
 - **deliberative subsystem**, containing a world model, which develops plans and makes decisions based on long-term predictions
 - **reactive subsystem**, which is capable of reacting to events without complex reasoning
- Often, the reactive component has some kind of precedence over the deliberative one. For example: low level collision avoidance behavior vs high level motion planning

Learning in autonomous agents

Machine learning techniques can support different components of deliberative, reactive, and hybrid agents:

- **Perception and representation:** extract state and semantic information from sensor data
- **World modeling:** learn predictive models of the environment, possibly in a latent representation space, to support planning
- **Decision making:** learn behaviors or policies from demonstrations (*imitation learning*) or through interaction with the environment (*reinforcement learning*)
- **Multi-agent learning:** train policies centrally using global information and execute them decentrally using local information (*CTDE*)
- **Foundation models:** use large pretrained language or vision models to support general-purpose perception, reasoning, and decision making

Practical autonomous agents often combine deliberation, reactivity, and learning within **hybrid architectures**.